PROCUREGUARD-AI: A NEUROSYMBOLIC PLATFORM FOR MSME TENDER COMPLIANCE AND PUBLIC PROCUREMENT INTEGRITY

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

by

AAKASH RAJ V (CB.AI.U4AID23106)
JEEVAKAMAL K R (CB.AI.U4AID23115)
DARANI KARTHIK R (CB.AI.U4AID23137)
SURIYA DHARSAUN K G (CB.AI.U4AID23144)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**ProcureGuard-AI: A Neurosymbolic Platform for MSME Tender Compliance and Public Procurement Integrity**" submitted by **Aakash Raj V., Roll. No: CB.AI.U4AID23106**, **Jeevakamal K R., Roll. No: CB.AI.U4AID23115**, **Darani Karthik R., Roll. No: CB.AI.U4AID23137** and **Suriya Dharsaun K G., Roll. No: CB.AI.U4AID23144** for the award of the **Degree of Bachelor of Technology** under the **Faculty of Engineering** is a bonafide record of the work carried out by them under my guidance and supervision at "Department of Artificial Intelligence", Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Premjith B.
Thesis Advisor,
Assistant Professor (Sr. Gd.),
Amrita School of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS

DECLARATION

We, **Aakash Raj V.**, **Reg. No: CB.AI.U4AID23106**, **Jeevakamal K R.**, **Reg. No: CB.AI.U4AID23115**, **Darani Karthik R.**, **Reg. No: CB.AI.U4AID23137** and **Suriya Dharsaun K G.**, **Reg. No: CB.AI.U4AID23144**, hereby declare that this thesis entitled "**ProcureGuard-AI: A Neurosymbolic Platform for MSME Tender Compliance and Public Procurement Integrity**" is the record of the original work done by us under the guidance of **Dr. Premjith B.**, Assistant Professor (Sr. Gd.), Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship or a similar award to any candidate in any University.

Place: **Signature of the Students**

Date:

COUNTERSIGNED

Dr. K.P Soman
Professor and Dean,
Department of Artificial Intelligence
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

Contents

Acknowledgements **iv**

List of figures **v**

List of tables **vi**

Abbreviations **vii**

List of symbols **viii**

Abstract **ix**

1 Introduction **1**

1.1 Background 1

1.2 Why Small Suppliers and Auditors Struggle 1

1.3 Problem Statement 2

1.4 Aim and Objectives 2

1.5 Research Questions 3

1.6 Scope, Authorship and Limitations 3

1.7 Contributions 4

1.8 Organisation of the Thesis 4

2 Literature Review **5**

2.1 Procurement Integrity and Small-Firm Participation 5

2.2 Language Models, Retrieval and Hallucination 6

2.3 Neurosymbolic Design 6

2.4 Collusion Detection on Networks 6

2.5 Forensic Accounting 7

2.6 Keystroke Dynamics 7

2.7 Multi-Criteria Bidder Selection 8

2.8 Comparison and Research Gap . . . 8

3 System Design **9**

3.1 Requirements . . . 9

3.2 Design Principle . . . 10

3.2.1 Alternatives Considered . . . 10

3.3 Architecture . . . 10

3.4 Roles and Access Control . . . 11

3.5 The Tender-Analysis Pipeline . . . 11

3.5.1 Walk-through of one request . . . 12

3.6 Data Model . . . 12

3.7 Security and Reliability . . . 13

3.8 Ethical and Social Considerations . . . 13

4 Document Engines: Compliance, Payment and Amendments **14**

4.1 Tender Compliance . . . 14

4.2 Statutory Payment Recovery . . . 16

4.3 Corrigendum Differ . . . 17

5 Auditor Engines: Collusion, Ledger, Behaviour and Trust **18**

5.1 Cartel Graph . . . 18

5.2 Ledger-Constrained Bank-Statement Verification . . . 20

5.3 Keystroke Bot Detection . . . 21

5.4 Bayesian Trust Optimizer . . . 22

6 Results and Discussion **24**

6.1 Evaluation Setup and Metrics . . . 24

6.2 Test Suites . . . 24

6.3 Keystroke Detector (RQ3) . . . 25

6.4 Ledger Repair (RQ2) . . . 27

6.5 Cartel Graph (RQ2) . . . 27

6.5.1 Sensitivity to the public-infrastructure cap . . . 28

6.6 Trust Optimizer Simulation . . . 29

6.7 Tender Pipeline (RQ1) . . . 29

6.8 Discussion and Threats to Validity . . . 29

7 Conclusion and Future Work **31**

7.1 Summary 31
7.2 Answers to the Research Questions 31
7.3 Limitations 32
7.4 Implications 32
7.5 Future Work 32

A Engine Constants **34**

B API Surface **36**

C Reproducing the Experiments **37**

References **38**

Acknowledgements

We thank our thesis advisor, Dr. Premjith B., for guidance throughout this project, and the Department of Artificial Intelligence at Amrita Vishwa Vidyapeetham for the facilities it provided. Each of us also thanks the other team members, whose modules together form the platform described here. We are grateful to our families and friends for their patience while this report was being written.

AAKASH RAJ V
JEEVAKAMAL K R
DARANI KARTHIK R
SURIYA DHARSAUN K G

List of Figures

3.1 Tiers of ProcureGuard-AI. 11

6.1 Share of adaptive-bot sessions flagged at risk ≥ 50, by session length (full model). 26

6.2 False pairs flagged by the engine as the public-infrastructure cap varies. 28

List of Tables

2.1 Related work and what it leaves open 8

3.1 Requirements derived from the problem statement 9
3.2 Nodes of the tender-analysis graph 12

5.1 Evidence weight $w(d)$ of a shared attribute held by d bidders 18
5.2 Feature groups used by the keystroke detector 21

6.1 Automated test results on the final repository 25
6.2 Full-model detection at risk ≥ 50 (95% Wilson intervals) 25
6.3 Detector comparison, full mode, same data and false-alarm budget 26
6.4 Statements flagged out of 500 per condition 27
6.5 Cartel experiment at the default cap: 300 pools, 19,440 honest bidder pairs . . . 28
6.6 Honest pairs flagged by the engine for different caps (naive method: 9,655) . . . 28

A.1 Constants used by the engines 35

B.1 Gateway areas and access 36

Abbreviations

CDS	–	Compliance Distance Score
DPDP	–	Digital Personal Data Protection
EM	–	Expectation–Maximisation
EMD	–	Earnest Money Deposit
GeM	–	Government e-Marketplace
GFR	–	General Financial Rules
GMM	–	Gaussian Mixture Model
JV	–	Joint Venture
JWT	–	JSON Web Token
LLM	–	Large Language Model
MAD	–	Median Absolute Deviation
MCP	–	Model Context Protocol
M2M	–	Machine-to-Machine
MSE	–	Micro and Small Enterprises
MSME	–	Micro, Small and Medium Enterprises
MSMED	–	Micro, Small and Medium Enterprises Development (Act)
OCD	–	Overall Compliance Distance
OCR	–	Optical Character Recognition
OECD	–	Organisation for Economic Co-operation and Development
RAG	–	Retrieval-Augmented Generation
RBI	–	Reserve Bank of India
ROC-AUC	–	Area Under the Receiver Operating Characteristic Curve
SDK	–	Software Development Kit

List of symbols

$R,\ A$	–	Required value and vendor's actual value of a rule
$w(d)$	–	Evidence weight of a shared attribute of degree d
$B_t,\ C_t,\ D_t$	–	Balance, credit and debit of ledger row t
$H_k,\ F_k$	–	Hold time and flight time of keystroke k
τ	–	Deadline-proximity multiplier
$B_{\text{ref}},\ B_{\min}$	–	Independent reference price of a tender; lowest admissible bid
$r,\ m$	–	Annual interest rate; number of monthly rests
ρ_i	–	Partner-sharing fraction of bidder i (Eq. 5.2)
u_i	–	Equipment uniqueness factor of bidder i (Section 5.4)
$S_a,\ S_o,\ S_c$	–	Asset, owner and network components of A_i
$s_{ik},\ f_{ik}$	–	Successful and failed projects of bidder i with client k
$S_{ik},\ w_k$	–	Score of bidder i from agent k and its weight in Z_i
$\mathbf{m}_j,\ J$	–	Mean of mixture component j; number of components
A_i	–	Anomaly score of bidder i
Z	–	Robust Z-score of the recent balance (Section 5.2)
$\delta,\ \alpha,\ \beta,\ \gamma,\ \mu$	–	Trust-optimizer price blend, Beta prior, legal and collusion weights
$\eta_j,\ C_{ij}$	–	Per-tender weight multiplier and confidence of agent j
T_i	–	Fused trust value of bidder i
Z_i	–	Final ranking score of bidder i

Abstract

Small suppliers in Indian public procurement lose bids on technicalities buried in tender documents of a hundred pages or more, wait months for statutory payments, and have no way to notice when a clause is quietly changed days before the deadline. Auditors face the opposite problem: they review tenders one at a time and cannot easily see cartels, doctored bank statements or scripted bidding. Handing these decisions to a large language model is unsafe, because the model can invent a number, gives different answers on different runs, and leaves no verifiable trail.

This thesis describes ProcureGuard-AI, a platform with a vendor portal and an auditor portal in which language models only read and explain text, while every eligibility, interest, solvency, collusion and award result is computed by deterministic, tested code (the only model labels that enter a score are those on corrigendum clauses). The report documents the engines behind each result: a retrieval and rule-extraction pipeline with a Compliance Distance Score, a compound-interest calculator for the MSMED Act, a corrigendum differ, a bipartite cartel graph with a public-infrastructure cap, a ledger verifier that separates scanning errors from tampering, a Gaussian-mixture keystroke detector, and a Bayesian trust optimizer.

All evaluation uses synthetic data. On simulated typing sessions the keystroke detector flags 4.6% of genuine humans, detects every regular script and burst injection, but only 54.6% of adaptive bots (ROC-AUC 0.958 over the three core attack types). In a controlled ledger experiment, character-confusion repair cleared 483 of 500 statements with scanning noise (96.6%) while still flagging all 500 tampered ones. In a cartel experiment the engine recovered all 120 planted rings but removed only 29.9% of false pairs at the default setting; a sweep of the public-infrastructure cap shows that this figure is a tuning choice, not a fixed property. The keystroke result also has a weak side: when one human typing style is withheld from training, 51–81% of its sessions are flagged. No real procurement data was available, so real-world accuracy remains unproven.

Keywords: public procurement, MSME, neurosymbolic AI, retrieval-augmented generation, collusion detection, bank-statement forensics, keystroke dynamics.

Chapter 1

Introduction

1.1 Background

Governments are among the largest buyers in any economy. The OECD estimates that public procurement accounts for roughly 12% of GDP across its member countries [1], and in India central and state bodies buy through the Government e-Marketplace (GeM), the Central Public Procurement Portal and a number of state e-procurement systems. Because the volumes are so large, procurement is also an instrument of policy. Three provisions matter most here. The Public Procurement Policy for Micro and Small Enterprises Order 2012 sets an annual purchasing target from small firms for central bodies [2]. The bid-security provisions of the General Financial Rules (GFR) 2017 allow registered micro and small enterprises to bid without paying the earnest money deposit (EMD) [3]. And Chapter V of the MSMED Act 2006 obliges a buyer to pay a micro or small supplier within 45 days of acceptance, failing which the buyer owes compound interest at three times the bank rate notified by the Reserve Bank of India [4].

On paper, then, a small supplier is well protected. In practice many of these protections go unused, because a small firm rarely has a lawyer or compliance officer, and the system also leaks integrity in ways that one clerk reading one tender cannot see.

1.2 Why Small Suppliers and Auditors Struggle

The project started from six recurring problems, listed below. Each engine described later was built to answer one of them.

1. **Document burden.** A tender document commonly runs to 100–150 pages, with the eligibility conditions spread over the notice, the general conditions and several annexures. A supplier can miss a mandatory certificate, or fail to notice that an exemption they are entitled to is mentioned only in an annexure.

2. **Late payment.** Working out what is owed under the MSMED Act means calculating compound interest with monthly rests and then writing a demand in the right form. Many

suppliers simply write off the delay.

3. **Late amendments.** A corrigendum published a day or two before the closing date can raise a financial threshold just enough to exclude a competitor, and nobody compares the amended text line by line with the original.

4. **Cartels.** Bidders who look independent may share directors, network addresses, bank branches or machinery, and a tender-by-tender audit does not reveal that overlap.

5. **Doctored statements.** A bank statement submitted as proof of solvency can be edited, spliced from several documents, or padded with a deposit that is withdrawn again after submission.

6. **Scripted bidding.** Software can submit or refresh bids faster and more regularly than any person can.

The first three problems fall mainly on suppliers and the last three on the people who oversee the process, which is why the platform has two portals (Chapter 3).

1.3 Problem Statement

Suppliers need help reading documents and enforcing their statutory rights. Auditors need relational and forensic tools, because looking at one tender at a time cannot expose patterns that appear only across bidders and across time.

The tempting response is to hand the whole task to a large language model (LLM): read the tender, judge the bidder, name the winner. We consider this unsafe for decisions that carry legal weight, for three reasons. First, generative models can state figures that do not appear in the source text [5], so a hallucinated threshold turns into a wrongful disqualification. Second, they are stochastic, so the same input does not always give the same output, and a decision that changes between runs is hard to defend in front of an administrative tribunal. Third, they do not guarantee arithmetic or accounting invariants, such as the rule that each balance in a bank statement equals the previous balance plus credits minus debits, or that compound interest is computed with the right number of rests.

The problem is therefore: *how can language models make tender documents readable without taking any decision that has to be correct, repeatable and traceable?*

1.4 Aim and Objectives

The aim is to design, build and evaluate ProcureGuard-AI, a platform that helps small vendors check tender eligibility and claim late-payment interest, and that gives auditors forensic tools for cartels, ledgers and bot bidding. Each objective below corresponds to a module in the project repository:

1. build an in-memory retrieval and extraction pipeline that turns a tender PDF into typed rules with source-page citations and confidence scores;
2. compute eligibility deterministically as a Compliance Distance Score and apply the GFR 2017 exemption;
3. implement the interest calculation of MSMED Act Sections 15–17 and draft the demand notice;
4. flag corrigendum clauses that changed meaning, and weight the risk by how close the change is to the deadline;
5. build a bipartite cartel graph that caps public-infrastructure attributes, discounts common attributes in its anomaly score and respects declared joint ventures;
6. verify bank statements as ledgers, separating scanning errors from tampering;
7. score keystroke timing for scripted behaviour without recording what was typed;
8. rank bidders with a Bayesian trust optimizer hardened against collusion, all behind one role-controlled API.

1.5 Research Questions

Three research questions guide the evaluation.

- **RQ1.** Does keeping the language model on the reading side, and all decisions in deterministic code, give results that are correct, repeatable and traceable?
- **RQ2.** Do the safeguards in the cartel and ledger engines (the public-infrastructure cap and character-confusion repair) reduce false alarms without hiding real manipulation?
- **RQ3.** Can a model trained only on genuine human typists separate scripted submissions from human ones using timing alone?

1.6 Scope, Authorship and Limitations

ProcureGuard-AI is a team project and this thesis describes the whole platform so that the pieces can be understood together. The repository records four contributors: Jeevakamal K R (compliance and legal-diff modules, API gateway, persistence, M2M gateway and integration), Darani Karthik R (access control, database hosting, the MCP server and the behavioural-biometrics engine), Aakash Raj V (solvency extraction and ledger verification) and Suriya Dharsaun K G (the cartel graph and the Bayesian trust optimizer). Each engine is described

from its code and tests, and the experiments reported in Chapter 6 were written by the team for this thesis.

The main limitations are stated here so that the reader can keep them in mind through the later chapters.

- No real procurement, typing or bank-statement data was available to us. Every experiment uses synthetic data generated with fixed random seeds.
- The payment clock in the interest calculator starts at the invoice date, whereas Section 15 of the Act starts it from acceptance or deemed acceptance.

1.7 Contributions

The thesis makes the following contributions.

- A pipeline in which model output is parsed into typed schemas and every later decision is computed by ordinary code, with a page citation attached to each extracted rule.
- A ledger verifier that tries character-confusion repairs before raising a tampering flag, together with an experiment showing that it does not excuse randomly inflated balances; the harder case of a forger using confusable digits is left untested.
- A cartel scorer that caps public-infrastructure attributes, discounts common attributes in its anomaly score, merges flagged pairs into rings and exempts declared joint ventures, together with a sweep that shows how sensitive it is to the public-infrastructure cap.
- A keystroke detector trained only on human sessions, with a reliability label on each score, and controlled experiments for RQ2 and RQ3 whose scripts regenerate the numbers in the tables (the RQ3 numbers come from the biometrics evaluation scripts).

1.8 Organisation of the Thesis

Chapter 2 reviews the research on which the platform rests and states the gap it addresses. Chapter 3 sets out the requirements, architecture, access model and data model. Chapter 4 describes the engines that read and compare documents (compliance, payment recovery, corrigendum differ) and Chapter 5 those used by auditors (cartel graph, ledger verifier, keystroke detector, trust optimizer). Chapter 6 reports the test results and experiments and discusses their validity. Chapter 7 answers the research questions and lists future work. The appendices list the engine constants, the API surface and the commands to reproduce the experiments.

Chapter 2

Literature Review

The platform draws on several research areas that rarely appear together: procurement studies, retrieval and language models, neurosymbolic design, network analysis, forensic accounting, keystroke biometrics and trust modelling. This chapter takes them in turn, says what each contributes to the design, and ends with the gap that the project tries to fill.

2.1 Procurement Integrity and Small-Firm Participation

Thai describes public procurement as a subject that was neglected for a long time and that sits between public administration, law and supply management [6]. Flynn and Davis later reviewed the research literature and found that only a minority of published procurement studies rest on an explicit theory [7]. For a practical project, that is a reminder that much of the useful knowledge sits in rules and procedures, which is why the platform encodes the Indian rules directly.

There is an economic argument for helping small firms to bid. Preparing a bid has a fixed cost. If the cost of reading a tender and proving compliance is larger than the expected profit multiplied by the probability of winning, a rational small firm stays out, and the buyer faces less competition [8]. Any tool that lowers the cost of reading a tender therefore does more than save time for the supplier; it widens the pool of bidders.

On the integrity side, the OECD treats procurement as a high-risk area for corruption and recommends transparency and data-driven oversight [1]. Fazekas and Kocsis build objective risk indicators from procurement records, such as contracts with a single bidder or the use of restricted procedures, and argue that corruption usually works through procedure rather than through open bribery [9]. An earlier index by Fazekas, Tóth and King follows the same logic with a different set of indicators [10]. This thesis applies the idea at the level of the document: a tailored specification or a late amendment is a procedural red flag that can be read from the tender text itself, without waiting for award data.

2.2 Language Models, Retrieval and Hallucination

A tender is too long to place in front of a language model in one piece, and even when the context window is large enough, the relevant clause may be buried in pages of boilerplate. Retrieval-augmented generation (RAG) addresses this by fetching the relevant passages first and giving only those to the model [11].

Retrieval can be lexical or dense. Lexical methods such as BM25 [12] match the words of the query and are precise for exact identifiers such as the abbreviation "EMD". Dense methods such as Sentence-BERT [13] embed passages as vectors and match meaning across different wording, so a query about "security deposit" can find a clause that says "earnest money". Each has a typical failure: lexical retrieval misses paraphrases, and dense retrieval can drift towards passages that are about the right topic but do not contain the number being sought. The platform uses dense retrieval with several narrow questions instead of one broad one (Section 4.1).

Retrieval narrows what the model sees but does not make the model reliable. The survey by Ji et al. shows that generation systems can still produce unsupported content, including wrong figures and reversed conditions [5]. For a legal threshold, one wrong digit is enough to decide a case wrongly. This is the central risk that the project is designed around.

2.3 Neurosymbolic Design

Neurosymbolic AI combines learned perception with explicit rules. Garcez and Lamb describe it as a way to obtain the flexibility of neural models together with the traceability of symbolic reasoning [14], and Marcus argues that robust systems need explicit structured knowledge and operations on variables alongside learning [15]. The design used in this thesis is the narrowest form of the idea. The neural component produces a typed record, the record is validated against a schema, and from that point only ordinary code touches it. There is no learned reasoning between the extraction and the decision, which means the decision can be unit-tested and replayed.

2.4 Collusion Detection on Networks

Bid rigging takes several forms, among them cover bidding, bid rotation and market allocation [8]. Wachs and Kertész model auction markets as networks of buyers and firms and show that the structure of co-bidding reveals cartel-like behaviour [16]. A naive version of this idea flags any two bidders that share an attribute. Independent firms do share things, though: a regional internet provider, a bank branch, a business park address. A practical engine therefore has to weaken the evidence that comes from common attributes, report rings and not isolated pairs, and respect legal joint ventures.

The weakening can be borrowed from link analysis. Adamic and Adar, working on social networks, weight a shared neighbour by the inverse logarithm of its degree, so that a feature held by everyone counts for little [17]. Work on bipartite projection in recommender systems reaches a similar conclusion, that a popular node should contribute less to the similarity of the items it connects [18]. The cartel engine uses the same inverse-logarithm weight (Section 5.1). The graph itself is built with NetworkX [19].

2.5 Forensic Accounting

Benford's law, which Benford documented across many datasets in 1938, says that in many collections of naturally occurring amounts the leading digit d appears with probability $\log_{10}(1 + 1/d)$ [20]; Nigrini's book is the standard guide to using the law in audits [21]. The test is weak on small samples and can only support suspicion, never prove manipulation.

A bank statement also obeys an exact rule: each balance equals the previous balance plus credits minus debits. That rule is stronger than any statistical test. The difficulty is that scanned statements read through optical character recognition (OCR) break it by accident, for example when a digit 1 is read as 7. A verifier therefore has to tell a scanning slip from a real edit, and the literature on Benford-style tests does not address that distinction because it assumes clean digital data.

Window dressing is the practice of parking money in an account shortly before it is examined. Detecting it means comparing recent balances with history, and robust scale estimators help here. The median absolute deviation [22] is not distorted by a few extreme values in the way a standard deviation is, so a single large deposit in the past does not hide a new one.

2.6 Keystroke Dynamics

Keystroke dynamics identifies a person or a behaviour from the timing of key presses. Monrose and Rubin showed early on that typing rhythm carries enough information to support authentication [23]. The two basic features are hold time (key down to key up) and flight time (key up to the next key down). Killourhy and Maxion compared anomaly detectors on a single benchmark and found that detector quality varies widely between algorithms [24].

For bot detection the setting differs from authentication in an important way: the attacks of tomorrow are not known today, so the detector should be one-class, trained on human behaviour only, in the sense surveyed by Chandola et al. [25]. Isolation Forest [26] and Gaussian mixture models [27], fitted by expectation–maximisation [28], both fit this pattern. Two design limits matter. Capturing which key was pressed would be a privacy hazard, so only timing should be stored. And short sessions carry little evidence, so a score must come with a reliability level and must never block anybody automatically.

2.7 Multi-Criteria Bidder Selection

Lowest-price award invites adverse selection, which is why many systems score price together with experience, finances and technical ability. Jøsang et al. survey trust and reputation models that combine several pieces of evidence into one score [29], and the beta reputation system gives a compact way to turn counts of successes and failures into an expected reliability [30]. Technical capacity can be modelled as a product of inputs in the manner of a production function [31].

Weighted scores are easy to game. A cartel can submit decoy prices to move the scale, inflate each other's track records, or hide lawsuits. A hardened selector must therefore anchor price to an independent estimate, weight evidence by how well it was verified, and penalise suspected collusion. Section 5.4 shows how the optimizer does each of these.

2.8 Comparison and Research Gap

Table 2.1 summarises the strands above and what each leaves open for this problem.

Table 2.1: Related work and what it leaves open

Area	**Representative work**	**What remains open here**
Retrieval-augmented generation	Lewis et al. [11]	No guarantee on numbers; no decision layer.
Neurosymbolic AI	Garcez and Lamb [14]	General position; few concrete procurement systems.
Corruption risk indicators	Fazekas and Kocsis [9]	Built on award data, not on tender documents.
Co-bidding networks	Wachs and Kertész [16]	Needs award history; no treatment of shared public infrastructure.
Benford and forensic tests	Nigrini [21]	Statistical only; ignores OCR noise.
Keystroke anomaly detection	Killourhy and Maxion [24]	Benchmarked for authentication, not bid submission.
Trust and reputation	Jøsang et al. [29]	Not hardened against collusive inputs.

Each area has good individual work. What we did not find in the literature we reviewed is a single working system that serves both sides of the market and holds one rule throughout: the language model reads, and code decides. We could not find a published system for Indian tenders that combines these pieces, but we cannot claim that none exists. ProcureGuard-AI is an attempt at that combination, evaluated honestly on the data that was available.

Chapter 3

System Design

The descriptions in this chapter and the next two were checked against the final `ProcureGuard-AI` repository (backend, `engines/` packages and frontend), not against earlier prototypes.

3.1 Requirements

The requirements come from the six problems in Chapter 1. Table 3.1 separates what the system must do from the qualities it must have. The non-functional requirements mattered more than the functional ones in practice, because they decided where the language model could and could not be used.

Table 3.1: Requirements derived from the problem statement

ID	Requirement	User
F1	Extract eligibility rules from a tender PDF with page citations	Vendor
F2	Compare the vendor profile with the rules and report the gap per rule	Vendor
F3	Compute statutory interest on a late payment and draft a notice	Vendor
F4	Compare an amendment with the original and rate the risk	Vendor, auditor
F5	Show collusion rings among bidders	Auditor
F6	Verify a bank statement and report a solvency figure	Auditor
F7	Score a bid session for scripted behaviour	Auditor
F8	Rank bidders with an explainable trust value	Auditor
N1	Every decision is reproducible: same input, same output	All
N2	Every extracted figure is traceable to a page	Vendor
N3	Role-based access, with reviewers separated from vendors	All
N4	No key text is stored by the behaviour SDK	Auditor, bidder
N5	Each engine can be run and tested without the others	Developer

3.2 Design Principle

The platform follows one rule: a language model may read and explain, but it never decides the outcome of an eligibility, interest, solvency, collusion or award computation. Two model outputs do feed a computation, and both are described where they occur: the invoice reader (Section 4.2) and the label that sets the base score of a corrigendum clause (Section 4.3). Gemini or Llama turns tender text into a typed record and later writes a plain-language summary. Eligibility, interest, solvency, collusion and award results come from ordinary Python that can be unit-tested and re-run. Requirement N1 (same input, same output) therefore holds from the typed record onwards; the extraction step itself depends on which provider answers (Section 4.1) and is made checkable through page citations and confidence scores, not made deterministic. Bank-statement figures are taken from the PDF text layer, not from a model, and when OCR is unavoidable the result is labelled `ocr_used` so that it cannot be mistaken for a text-layer reading.

3.2.1 Alternatives Considered

Two simpler designs were considered and rejected.

The first is an end-to-end LLM pipeline in which the model reads the tender and the vendor profile and returns a verdict. It is the fastest to build, but it breaks requirements N1 and N2: the verdict can differ between runs, and there is no step at which a wrong threshold can be caught. The second is a pure rule-based system with hand-written parsers for tender layouts. This satisfies N1 but copes badly with real documents, because tender layouts vary between portals and departments and every new layout needs a new parser. The adopted design splits the task at the point where each approach is strong. Reading unstructured text is left to the model, and everything after the typed record is plain code.

3.3 Architecture

Figure 3.1 shows the four tiers. The engines for the cartel graph, solvency, biometrics and trust optimizer are independent Python packages under `engines/`. The gateway imports them, and each can also run alone as its own small FastAPI service and be tested on its own (requirement N5).

The stack is Python 3.12 or later (the tests in Chapter 6 ran on 3.13) with FastAPI, SQLAlchemy 2.0, LangGraph and Pydantic; PyMuPDF, Chroma and MiniLM embeddings for retrieval; NetworkX, pandas, NumPy and scikit-learn in the engines [19, 32, 33]; and Next.js 16.3 with React 19.2, Tailwind CSS 4 and D3 in the interface. The gateway registers 14 routers in `backend/src/main.py` (authentication, MSME profiles, legal diff, cartel, behaviour, solvency, matchmaker, notifications, tenders, documents, M2M, jobs, trust optimizer and vendor bidding) and mounts the MCP server. The web interface is a Next.js application with 14 page routes

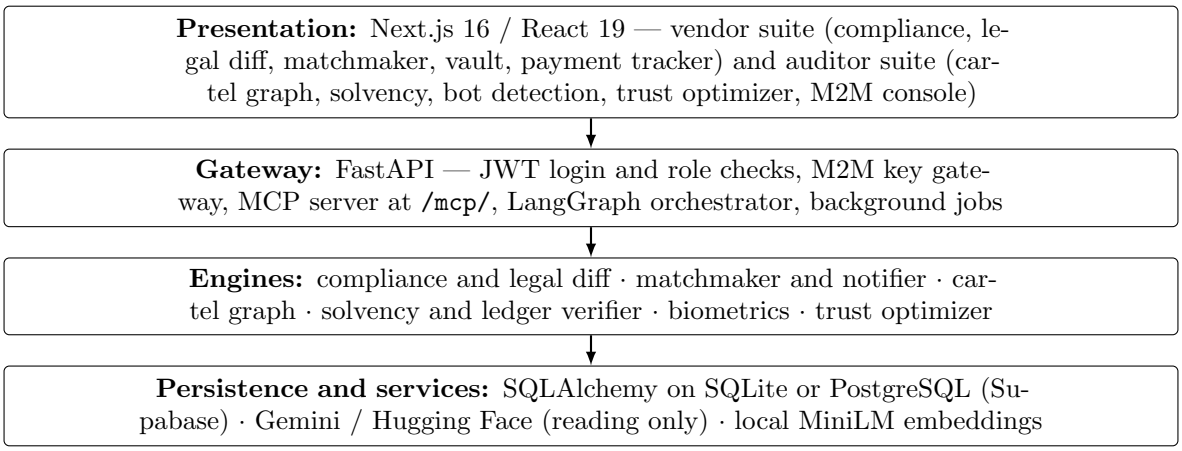


Figure 3.1: Tiers of ProcureGuard-AI.

plus a landing page.

The repository also keeps the same code split into two applications, `frontend-client` (vendors, port 3000) and `frontend-admin` (auditors, port 3001). In the split vendor build the payment-tracker page is only a redirect, while the unified application has the full page; the thesis treats the unified application as the reference.

3.4 Roles and Access Control

Three roles exist: `msme` (a vendor), `government_auditor` and `admin`. The policy sits in `backend/src/core/access.py` and is applied per router. Reviewers (auditor and admin) alone reach the cartel graph, solvency, trust optimizer, the bot-detection review queue and the MCP server. Any signed-in user can analyse tenders and run legal diff. The matchmaker, notifications and documents are limited to the owning enterprise plus reviewers, and the M2M gateway uses an API key. A test in `test_access_control.py` fails if any route is left unguarded, so a new endpoint cannot be added without a policy decision. The reason for such a test is that the easiest access-control mistake in a growing API is to forget a guard on a new route, and a test that enumerates the routes catches it automatically.

3.5 The Tender-Analysis Pipeline

Tender analysis is a LangGraph state graph of six nodes that share one typed state object. The graph runs in a fixed order and stops at the first failing node, returning that node's own error. Table 3.2 lists the nodes; only nodes 3, 5 and 6 use a model.

The graph is reachable through `POST /api/orchestrator/analyze`, or as a background job through `POST /api/jobs/tender-analyze`, which returns a job identifier that the client polls until the job is done. Section 4.1 gives the method of nodes 2–4.

Table 3.2: Nodes of the tender-analysis graph

#	Node	Job	Model?
1	`pdf_extractor`	Text from the PDF with PyMuPDF	No
2	`rag_retriever`	Chunk, embed, keep the clauses that matter	No
3	`neural_extractor`	Clauses to typed rules with confidence and page	Reads
4	`symbolic_verifier`	Compare rules with vendor profile, compute CDS	No
5	`tender_simplifier`	Plain and regional-language summary	Explains
6	`scheme_discoverer`	Suggest support schemes for the gaps	Suggests

3.5.1 Walk-through of one request

It helps to follow a single request through the tiers. A vendor signs in and the interface stores a JWT. The vendor uploads a tender PDF and presses analyse. The gateway checks the token and the role, creates an asynchronous job record and returns its identifier. A worker runs the six nodes in order. Node 1 reads the PDF text layer; node 2 builds an in-memory index and retrieves clauses; node 3 asks a model for typed rules; node 4 compares them with the vendor's stored MSME profile and computes the scores; nodes 5 and 6 add the summary and scheme suggestions. The job record is updated with the result, and the interface, which has been polling, renders the per-rule table with its traffic-light statuses and page citations. At no point after node 3 does a model change a figure or a decision. Nodes 5 and 6 produce free text only, and the platform does not yet check that text against the typed record, so a summary could still misquote a figure; the page-cited rule table is the authoritative output.

3.6 Data Model

SQLAlchemy 2.0 maps twelve tables: `users`, `msmes`, `user_enterprises` (which enterprises a user may act for), `tenders`, `tender_bids` (bids placed through the vendor portal, which are the live input to the cartel and trust engines), `documents`, `async_jobs`, `m2m_clients`, `match_notifications`, `behavior_signals`, `behavior_review_events` and `cartel_incident_reviews`.

Two choices in this schema are worth explaining. Behaviour signals are stored as measurements, and each reviewer decision is a separate row in an append-only event table, so a review never changes the original score; an auditor can later see both what the detector said and what the human decided. And the user-to-enterprise link is its own table, which lets one login act for more than one enterprise. The database defaults to a local SQLite file and switches to PostgreSQL, including Supabase with row-level security, when `DATABASE_URL` is set.

3.7 Security and Reliability

- Passwords are hashed with bcrypt (12 rounds). Login tokens are HS256 JWTs valid for 24 hours. In production mode the server refuses to start without a signing secret and creates no demo accounts.
- The M2M gateway checks the `X-ProcureGuard-Key` header and is switched off if no key is configured. Signed requests (HMAC) are not yet verified on the server.
- The MCP server exposes bot-detection tools (analyse timing, list and read signals, model status), all read-only, to signed-in auditors and admins, with DNS-rebinding protection. An assistant connected through MCP cannot record or change a review.
- The keystroke SDK sends only timings, never characters (Section 5.3). This is consistent with the data-minimisation expectations of the Digital Personal Data Protection Act 2023 [34], but this thesis does not make a legal compliance claim.
- Vector indexes are built in memory per request and discarded. A background notifier scans tenders every 300 seconds by default and writes at most one notification per vendor, tender and kind, enforced by a unique constraint.

3.8 Ethical and Social Considerations

Three of the engines make statements about people or firms, and a false statement has a cost. A wrongly flagged bidder may lose business; a wrongly flagged typist may be treated as a bot. The design responds in four ways. First, nothing blocks anyone automatically: cartel, ledger and behaviour results go to a human reviewer as leads. Second, every result states its own reliability, as the keystroke score does through its low, medium and high labels. Third, the biometric input is limited to timings and no characters are recorded, which makes it much harder to recover what a person typed; timing is still treated as personal data. Fourth, because typing style differs for people who use assistive technology, the evaluation in Chapter 6 reports the effect of withholding a typing style from training and does not hide it.

There is also a question of who benefits. A tool that helps small suppliers read tenders could, in principle, also help a dishonest supplier find the weakest requirement to game. The platform does not guard against that, and we do not think it can; the countermeasure is the auditor-side tooling, which is why both portals exist.

Chapter 4

Document Engines: Compliance, Payment and Amendments

This chapter and the next give the method of each engine, together with the constants that the code uses. File names are relative to the repository root. The three engines here work on documents, mainly for vendors: they read a tender, compute what a late payment is worth, and compare an amendment with its original.

4.1 Tender Compliance

Retrieval. The tender text (`agents/compliance/rag_retriever.py`) is cut into chunks of at most 1500 characters with 200 characters of overlap, using a recursive splitter that prefers paragraph breaks. The overlap is there so that a sentence cut at a chunk boundary still appears whole in one of the two neighbours. Every chunk is embedded with the local `all-MiniLM-L6-v2` model into a 384-dimensional vector and stored in an in-memory Chroma index.

A single broad question tends to return clauses that are generally about tenders and miss the eligibility table, so five separate questions are asked (EMD, turnover, experience, MSE/startup relaxation, mandatory certificates). Each returns its four nearest chunks by cosine similarity, and the results are merged without duplicates. Retrieval is dense only; no lexical (BM25) component is used.

Extraction. The merged text goes to `neural_extractor.py`. If a Hugging Face token is set, Llama-3.1-8B-Instruct is tried first; if that fails, Gemini is tried; if that also fails, a regular-expression parser runs so that the pipeline still returns something. The model output is parsed into a Pydantic `TenderRequirements` object holding the EMD amount, required turnover, required experience, certificates, an MSME-exemption flag, and for each rule a confidence in $[0, 1]$ and a source page. The temperature is 0.1, which keeps the output close to deterministic without relying on it being so.

One weakness deserves mention. When the regular-expression fallback finds nothing, it fills

in default values (EMD 50,000, turnover 25 lakh, three years), so a fallback result must be read with care. Silently supplying thresholds also runs against the principle of Chapter 3 that every figure should be traceable to a page. A cleaner design would return an explicit "not found", and this is listed under future work.

Verification. From here no model is involved (`symbolic_verifier.py`). For a numeric rule with required value R and vendor value A, the Compliance Distance Score is

$$\mathrm{CDS} = \frac{A - R}{R} \times 100, \tag{4.1}$$

which is positive when the vendor exceeds the requirement and negative when it falls short; it is 100 when there is no requirement. The score is a percentage gap, so the same number means the same thing for a rule in rupees and a rule in years.

If the tender grants an MSME relaxation and the vendor is a registered MSME, the EMD and turnover rules receive the status `EXEMPTED`. The EMD exemption follows GFR 2017 [3]; the code applies the same flag to turnover, a relaxation that comes from the MSE procurement policy and that should be checked against each tender's own wording. An overall score combines the rules with fixed weights w (EMD 3, turnover 2, experience 2, certificates 1):

$$\mathrm{OCD} = \frac{\sum_i w_i \, n_i}{\sum_i w_i}, \qquad n_i = \begin{cases} 100 & \text{pass or exempted} \\ \max(0,\, 100 + \mathrm{CDS}_i) & \text{fail.} \end{cases} \tag{4.2}$$

Each rule also carries a traffic-light status and a human-readable reason, so the result can be traced from the report back to the clause and page.

Worked example. Take a tender that requires a turnover of Rs 25 lakh and three years of experience, grants the MSE relaxation (which the code applies to EMD and turnover together), and asks for one certificate. A registered micro enterprise has a turnover of Rs 40 lakh, two years of experience and the certificate. The turnover rule gives $\mathrm{CDS} = (40-25)/25 \times 100 = +60$, which is still reported, but its status is exempted. The experience rule gives $\mathrm{CDS} = (2-3)/3 \times 100 \approx -33.3$ and fails, so its contribution is $n = 100 - 33.3 = 66.7$. The EMD rule is also exempted and the certificate passes, so all three contribute 100. Then

$$\mathrm{OCD} = \frac{3(100) + 2(100) + 2(66.7) + 1(100)}{3 + 2 + 2 + 1} = \frac{733.3}{8} \approx 91.7. \tag{4.3}$$

The report would show one amber row (experience, 33% short) and three green rows, and the vendor sees exactly which clause to address. The failure does not disappear into an average, because each rule keeps its own status next to the overall score. Two caveats apply. An exempted rule counts as 100, so an exempt vendor receives the full weight of those rules for free; dropping exempted rules from both sums would be a fairer score. And OCD is a gap measure, not a verdict: a failed mandatory rule is usually a disqualification in practice, so the per-rule

status, and not the average, is what the vendor should act on. The exemption is also decided by a single registered-MSME flag, whereas the GFR relaxation itself covers micro and small enterprises only, so a medium enterprise would be wrongly exempted.

Failure modes. The engine can go wrong in three places. The retriever may miss the clause, in which case the extractor has nothing to read. The extractor may read a figure wrongly, for instance taking the turnover of the previous year for the required turnover. And the vendor profile may be out of date. The first two are the reasons that every rule carries a page number and a confidence: the vendor can open the page and check. The third is outside the engine's control.

4.2 Statutory Payment Recovery

Section 15 of the MSMED Act sets the payment period at 45 days, and Section 16 makes the buyer liable for compound interest with monthly rests at three times the RBI bank rate [4]. The module (`payment_tracker.py`) uses a bank rate of 6.5%, so $r = 19.5\%$ a year. With D the days from invoice to payment, the overdue days are $D - 45$ if that is positive, and the number of monthly rests is $m = \lceil (D - 45)/30 \rceil$. The amount owed is

$$A = P\left(1 + \frac{r}{12}\right)^{m}, \qquad I = A - P. \tag{4.4}$$

Worked examples. An invoice of Rs 5,00,000 paid 90 days after the invoice date is 45 days overdue, so $m = 2$ and

$$I = 500000\left[(1.01625)^2 - 1\right] \approx \text{Rs } 16{,}382. \tag{4.5}$$

A smaller invoice of Rs 2,00,000 paid after 120 days is 75 days overdue, so $m = \lceil 75/30 \rceil = 3$ and $I = 200000\,[(1.01625)^3 - 1] \approx$ Rs 9,909. The monthly rate is $19.5/12 = 1.625\%$ in both cases.

Assumptions. Rounding part-months up slightly overstates the interest, and measuring from the invoice date rather than from acceptance is a known simplification. The bank rate is a constant in the code and would need to be updated when the RBI changes it, so the figures above are illustrative at a 6.5% rate; we did not check that value against the rate currently notified by the RBI, and it is not a statement of the current statutory rate. The invoice itself is read by a vision model, but only the extracted principal and dates enter the formula, and the vendor confirms them before a draft demand notice is produced. A wrong date therefore costs the vendor a correction, not a wrong legal claim.

4.3 Corrigendum Differ

Both documents are split into clauses and embedded with the same MiniLM model. For each original clause the most similar amended clause is found by cosine similarity, and the pair is flagged when that best similarity is below 0.75 (`semantic_differ.py`). Comparing by embedding and not by character difference means that a reworded but equivalent clause is not flagged. The method has two weaknesses that Chapter 6 does not evaluate systematically. Sentence embeddings change very little when only a number is altered. In a quick check on hand-written clause pairs (not a systematic evaluation), raising a turnover requirement from Rs 2 crore to Rs 5 crore gave a cosine similarity of 0.96, raising an EMD tenfold gave 0.95, and moving a deadline gave 0.95; even a pair that raised the experience requirement from three to ten years and added a new condition scored 0.81. All of these stay above 0.75 and would go unflagged. The clause pairs are not part of the repository's experiment scripts, so these four values are indicative only and cannot be reproduced from this thesis. And because the search runs from the original clauses, a wholly new clause has nothing to match against. A numeric and date comparison alongside the embedding, and a search in both directions, are the obvious remedies and are listed under future work.

A model then labels each flagged pair `RESTRICTIVE`, `PERMISSIVE` or `NEUTRAL`; this is the only model step, and because the label sets the base score it does influence the verdict. The rule is therefore that the model reads and labels while code computes, not that the model has no effect. The rest is arithmetic (`corruption_risk_scorer.py`). Each restrictive clause adds 0.3 and each neutral clause 0.1; the sum is capped at 1. A time multiplier τ is 3.0 if the amendment is at most 2 days before the deadline, 1.5 if at most 7 days, and 1.0 otherwise. The manipulation probability is

$$p = \min\big(0.99,\ \tau \cdot \min(1, \textstyle\sum_k b_k)\big), \tag{4.6}$$

where b_k is the base score of flagged clause k. A result is `CRITICAL` for $p \geq 0.75$, `SUSPICIOUS` for $p \geq 0.40$ and `SAFE` otherwise. The multiplier is a step function.

Worked example. Suppose an amendment changes one clause and the model labels it restrictive, so $\sum_k b_k = 0.3$. The risk depends on timing alone:

Amendment issued	τ	p	**Verdict**
10 days before deadline	1.0	0.30	`SAFE`
6 days before deadline	1.5	0.45	`SUSPICIOUS`
1 day before deadline	3.0	0.90	`CRITICAL`

The same clause change is treated as harmless early and as critical at the last moment, which is the behaviour intended: a restrictive change that leaves bidders time to respond is ordinary, and one that does not is a warning sign. The step function makes the thresholds easy to explain to a reviewer, at the cost of a jump at 2 and 7 days.

Chapter 5

Auditor Engines: Collusion, Ledger, Behaviour and Trust

The four engines in this chapter serve auditors. They work on relations between bidders, on the internal consistency of financial documents, on timing signals from the bid form, and on the ranking of bidders. Each of them produces a lead for a human reviewer, not a verdict.

5.1 Cartel Graph

Structure. The graph (`engines/cartel_graph/graph.py`) is bipartite: one side holds bidders, the other holds shared attributes of four kinds (directors, IP addresses, bank routing codes, and declared assets or certificates). Blank or missing values never become nodes, so two bidders with nothing recorded cannot match each other. This is a small rule but it matters, because missing data is common and treating "empty" as a shared value would link unrelated firms.

Evidence weight. An attribute node with degree $d \geq 2$ becomes a piece of evidence with weight

$$w(d) = \min\left(1,\ \frac{1}{\ln(1+d)}\right). \tag{5.1}$$

This is the inverse-logarithm weight used by Adamic and Adar for shared neighbours in social networks [17]: the more bidders share an attribute, the less it says about any two of them. Table 5.1 gives some values.

Table 5.1: Evidence weight $w(d)$ of a shared attribute held by d bidders

d	2	3	4	5	6	10
$w(d)$	0.910	0.721	0.621	0.558	0.514	0.417

The decay is slow, so the discount is mild and not strong. For stronger protection, an IP or bank code shared by more than five bidders (the default cap, `AEGIS_PUBLIC_DEGREE_CAP`) is

treated as public infrastructure and can never flag anyone on its own. A shared director is not subject to the cap, since a real person sitting on many boards is exactly what an auditor wants to see. Bidders that appear together in a declared joint-venture group are exempt from each other.

Pair flags. A pair of bidders is flagged when they share at least one attribute that is not public infrastructure and is not covered by a declared joint venture; there is no separate score threshold. The weight $w(d)$ does not decide the flag. It scales how much each shared attribute adds to a bidder's anomaly score below.

Rings. Flagged pairs are merged into rings by taking connected components of a bidder–evidence graph, so five colluding firms appear as one ring and not as ten pairs. For an auditor this changes the question from "which pairs look related?" to "which groups look coordinated?", which is closer to how a cartel is actually described in an enquiry. Because rings are connected components, a chain of weak links (A and B share an IP, B and C share a bank code) can merge unrelated bidders into one ring, and the auditor should read a ring as a lead and not as a finding.

Anomaly score. Each bidder also receives an anomaly score (`scoring.py`). Let $\bar{a}$, $\bar{o}$ and $\bar{n}$ be the weighted counts of shared assets, shared owners and shared network attributes, divided by the bidder's declared assets, owners and two respectively. With clamping to $[0, 1]$,

$$A_i = \text{clamp}\big(0.45\, S_a + 0.35\, S_o + 0.20\, S_c - 0.30\, \rho_i\big), \tag{5.2}$$

where $S_a = \text{clamp}(1.6\bar{a})$, $S_o = \text{clamp}(1.8\bar{o})$, $S_c = \text{clamp}(0.75\bar{n} + 0.75\bar{a})$ and $\rho_i = \min(1, 0.8 \cdot$ partners sharing assets/declared assets).

The sign of the ρ_i term is worth noting: the code subtracts it, although its comment says it should raise suspicion. Because shared assets already enter S_a and S_c, the subtraction acts as a calibration term. We report the behaviour as implemented and list the question under future work. Read literally, it means that a bidder whose partners share its assets receives a lower score than it otherwise would, which is the opposite of what a collusion detector should do, so the term needs either a code fix or a stronger justification.

Cost. Building the graph is linear in the number of bidder–attribute edges. The pair search compares every pair of bidders, so it is quadratic in the number of bidders in a pool. For the pools of a dozen or a few dozen bidders that a single tender attracts this is negligible, but it would need an index over attribute nodes before it could be run across a whole portal.

5.2 Ledger-Constrained Bank-Statement Verification

The verifier (`engines/solvency/ledger_verifier.py`) treats a statement as a ledger and applies five constraints, with a tolerance $\epsilon = 0.05$ rupees. The first is the strongest, because it is an identity and not a statistical test.

1. Row recurrence and OCR repair. For each row, $|B_t - (B_{t-1} + C_t - D_t)| \leq \epsilon$. If a row fails, the verifier generates candidates by replacing one digit of the integer part of the balance, the credit or the debit with a visually confusable digit (1/7, 3/8, 8/0, 5/6, and the corresponding letter confusions), and searches the combinations. If some combination satisfies the recurrence the row is marked `CORRECTED_OCR_ERROR` and the repaired balance is carried forward; otherwise it is an `UNEXPLAINABLE_RESIDUAL` and raises a critical flag.

Illustration. Suppose a row opens at Rs 95,000 with a credit of Rs 23,000, so the balance should be Rs 1,18,000. If the scan reads 7,18,000, the recurrence fails; changing the 7 to a 1 restores it, so the row is repaired and the correct balance is carried forward. If the document instead shows Rs 1,68,000, as it would after a Rs 50,000 insertion, no single confusable swap in the balance, credit or debit satisfies the recurrence, and the row is flagged as unexplained.

A forgery could in principle coincide with a confusion swap and be mistaken for scanning noise. The experiment in Section 6.4 measures how often that happens.

2. Page continuity. The opening balance of page $p + 1$ must equal the closing balance of page p, which exposes deleted or spliced pages.

3. Window dressing. A robust Z-score compares the mean balance of the last 30 days with the earlier history:

$$Z = \frac{\overline{B}_{30} - \text{median}(B_{\text{prior}})}{1.4826 \cdot \text{MAD}(B_{\text{prior}})}, \tag{5.3}$$

flagged when $Z \geq 3$. The constant 1.4826 makes the MAD comparable to a standard deviation for normally distributed data, and the median and MAD resist single outliers [22].

4. Round trips. A credit of at least twice the median credit is matched to a debit within the next 7 days whose amount differs by at most 2% and whose narration shares more than one token with the credit. This catches the pattern in which money enters shortly before submission and leaves again shortly after.

5. Benford check. With at least 30 amounts, the largest gap between the observed first-digit proportions and $\log_{10}(1 + 1/d)$ [20, 21] is computed, and a low-severity flag is raised if it exceeds 0.25. This is a maximum-deviation statistic, not a full Kolmogorov–Smirnov test, and it only signals suspicion. The value 0.25 is a default that was not tuned or validated here; it is lax, so only strongly non-Benford amounts would be flagged.

The solvency figure reported to auditors is a time-weighted average balance, so a one-day spike has little effect on it.

5.3 Keystroke Bot Detection

Capture and features. The browser SDK records only key-down and key-up times. Hold time is $H_k = t_k^{\mathrm{up}} - t_k^{\mathrm{down}}$ and flight time is $F_k = t_k^{\mathrm{down}} - t_{k-1}^{\mathrm{up}}$. Flights are clipped at 2000 ms and holds at 1000 ms so that a pause to read the form cannot dominate, and the first flight (the delay before typing starts) is discarded.

The features (the feature module of the biometrics engine) fall into four groups, shown in Table 5.2. All are defined so that they do not grow with the number of keystrokes, but their estimates are noisier for short sessions, which is why every score carries a reliability label.

Table 5.2: Feature groups used by the keystroke detector

Group	Content
Location and spread	Means, spreads and percentiles of log hold and log flight times
Shape	Skewness and kurtosis of the log timings
Rhythm	Lag-1 autocorrelation of the flight series
Interval mix	Share of short, long and pause intervals

Model. The log features are modelled by a Gaussian mixture [27]

$$p(\mathbf{x}) = \sum_{j=1}^{J} \pi_j \, \mathcal{N}(\mathbf{x} \mid \mathbf{m}_j, \mathbf{\Sigma}_j), \tag{5.4}$$

fitted by expectation–maximisation [28] on genuine human sessions only, with $J = 2$ components for the full mode (hold and flight) and $J = 4$ for the flight-only fallback; $\mathbf{m}_j$ and $\mathbf{\Sigma}_j$ are the mean and covariance of component j. A mixture is used because human typists are not one population: a fast typist and a slow one form two clusters, and a single Gaussian would call the space between them normal. These sizes were selected on a separate development dataset (seed 7) as the simplest candidate within 1% of the best macro detection rate at a human false-alarm rate near 5%.

The negative log-likelihood is mapped to a 0–100 risk score using a threshold calibrated per session-length band on a held-out human split, so that 50 is meant to be the 95th percentile of humans of similar length and 75 the 99th (Section 6.3 shows that the realised false-alarm rate still varies with length). A session is moderate risk from 50 and high risk from 75. A reliability label accompanies every score: low below 30 keystrokes, medium from 30 to 99, high from 100. Models are exported as JSON and scored with NumPy alone [32], so no training library is needed at the point of use.

Use. The score is advisory. Sessions that need review enter a pending queue, each reviewer decision is stored as its own event, and nothing ever blocks a bid automatically, because people who use assistive technology type differently from the training population.

5.4 Bayesian Trust Optimizer

For a tender with reference price B_{ref} (the independent estimate), five agents score each bidder (`engines/trust_optimizer`).

$$P_i = \delta \frac{B_{\min}}{B_i} + (1-\delta)\frac{B_{\text{ref}}}{B_i}, \quad \delta = 0.5, \tag{5.5}$$

$$\text{Perf}_i = \frac{\sum_k v_k s_{ik} + \alpha}{\sum_k v_k (s_{ik} + f_{ik}) + \alpha + \beta}, \quad \alpha = \beta = 1, \tag{5.6}$$

$$L_i = \exp\Big(-\big[\textstyle\sum_c n_{ic}^{\text{ver}} w_c + \gamma D_i\big]\Big), \quad \gamma = 2, \tag{5.7}$$

where n_{ic}^{ver} is the number of verified cases of category c for bidder i and w_c is the weight of that category.

Price. Bids more than 40% from B_{ref} are excluded when finding $B_{\min}$, so shell bids cannot move the price scale. Bids that far below B_{ref} therefore cannot lower the scale, and the eligibility gate is intended to stop them from winning on price alone (the simulation of Chapter 6 shows one such case, not a guarantee). The price score itself is a ratio, so a very low bid that passes the gate still scores higher; capping P_i at 1 would remove this and is listed under future work.

Performance. The performance score is a Beta-style estimate in the manner of the beta reputation system [30], in which s_{ik} and f_{ik} are the successful and failed projects of bidder i with client k. Each is weighted by a verification weight v_k, so that mutual-reference rings count for less. The prior $\alpha = \beta = 1$ pulls a bidder with no history towards 0.5 and not towards either extreme.

Legal. In the legal score, D_i counts audited cases above self-declared cases in each category, so under-declaring litigation is penalised.

Financial and technical. The financial score scales the product of liquidity, credit rating and profitability between a floor and a ceiling, after an eligibility gate on registration age (two years) and turnover. The technical score is a Cobb–Douglas product [31] of equipment, staff, technology and experience ratios, multiplied by a uniqueness factor $u_i = 1 - |\text{assets}_i \cap \text{assets}_{-i}|/|\text{assets}_i|$ that reduces the score when equipment is shared with other bidders. The price score is not bounded by 1; it reached 2.5 in the simulation.

Fusion and selection. A fused trust value $T_i = \sum_j C_{ij}\eta_j T_{ij} / \sum_j C_{ij}\eta_j$ combines the agent scores using confidences C (project and case counts) and multipliers $\eta_j \in [0.85, 1.15]$ drawn from a generator seeded by the tender identifier, so weights are reproducible for a given tender. Because the identifier is public, this is variation and not secrecy: it stops one fixed set of weights from being tuned against, but anyone with the code can recompute it. Selection removes bidders failing the gate or the minimum thresholds (each jittered by up to ± 0.02 per tender, which like the weights is recomputable from the public tender identifier and so adds variation, not security), then ranks the rest by

$$Z_i = \sum_k w_k S_{ik} - \mu A_i, \qquad \mu = 0.3, \tag{5.8}$$

where S_{ik} is the score of bidder i from agent k, w_k is the weight of that agent, and $A_i = (S_a + O_i + S_c)/3$ averages shared-asset share, ownership overlap and score clustering (another bidder within Euclidean distance 0.05), or is replaced by the cartel engine's score for real bids. If the top two values differ by less than 0.02 and either bidder has $A \geq 0.10$, the award is held for manual review. After the project, trust is updated as $T^{\text{new}} = 0.8\, T^{\text{old}} + 0.2\, o$ for outcome $o \in [0, 1]$, so one project moves the value by at most 20% of the gap.

Chapter 6

Results and Discussion

Every number in this chapter comes from a file or a command in the repository, or from one of the scripts `thesis/experiments/rq2_experiments.py` and `thesis_amrita/experiments/cap_sweep.py`. All data are synthetic. Where a result is weak we say so in the same paragraph that reports it.

6.1 Evaluation Setup and Metrics

Since no real procurement or typing data was available, each engine is evaluated on data produced by a generator with a fixed random seed. This makes every experiment repeatable, but it also means that the results describe the engine under the generator's assumptions. The measures used are the following.

- **False-alarm rate**: the share of genuine cases that are flagged. For the keystroke detector this is the share of human sessions with risk at or above 50.
- **Detection rate**: the share of attack cases that are flagged. The macro detection rate is the unweighted mean over the three core attack types, so that no single attack dominates.
- **ROC-AUC**: the probability that a randomly chosen attack session receives a higher risk score than a randomly chosen human session [35]. It summarises ranking quality independent of any threshold.
- **Wilson interval**: the 95% confidence interval for a proportion, used for the detection and false-alarm rates because it behaves sensibly near 0 and 1 [36].

6.2 Test Suites

We ran the suites on the final repository (Windows, Python 3.13). Table 6.1 gives the outcome.

The five backend failures are one test in `test_rule_retrieval.py` and four in `test_vendor_portal.py`. The first expects an error when both language-model providers fail,

Table 6.1: Automated test results on the final repository

Suite	Result
Cartel, solvency, trust optimizer	21 passed
Backend gateway	162 passed, 5 failed, 4 skipped
Biometrics engine	219 passed (the browser test and two modules needing `pytest-asyncio` were excluded from this run)

but the extractor now falls back to the regular-expression parser, so the test is out of date. In the other four, which concern the shipped bidder pool and IP fields, the cause was not diagnosed; they may depend on state in the local database. We report them as failures and do not hide them.

6.3 Keystroke Detector (RQ3)

The protocol is in `engines/biometrics/reports/evaluation_report.md` [37]. A generator with seed 42 produced 39,000 sessions of 10–300 keystrokes, with lengths drawn log-uniformly so that short and long sessions are both common. Genuine humans (typical, fast and slow typists) were split 60/25/15 into training, calibration and test by session; bots appear only in evaluation. The four bot classes differ in how closely they imitate a person. A regular script types at a perfectly even rhythm. Burst injection sends small chunks of characters separated by pauses. The adaptive bot varies its timing to look human, and its hold times are the main thing that betrays it (see the flight-only result below). The mimic bot is a stress test that copies the mean, spread and pause rate of human typing. Table 6.2 shows the full model.

Table 6.2: Full-model detection at risk ≥ 50 (95% Wilson intervals)

Class	Sessions	Flagged
Genuine humans (all three styles)	2700	4.6% (3.8–5.4)
Regular script	6000	100.0% (99.9–100)
Burst injection	6000	100.0% (99.9–100)
Adaptive bot	6000	54.6% (53.4–55.9)
Mimic bot (stress test)	3000	44.7% (42.9–46.5)
Macro detection (three core attacks)		84.9%
ROC-AUC (core attacks vs. humans)		0.958 (0.955–0.960)

Detection of the adaptive bot depends strongly on session length (Figure 6.1): 13% for 10–19 keystrokes, 40% for 30–49, 92% for 100–199 and 98% for 200–300. The figure shows four of the six length bands; the other two, 20–29 and 50–99 keystrokes, give 24% and 68%. A short session simply carries too little evidence, and it also raises false alarms on humans: 7.5% of genuine sessions of 10–19 keystrokes are flagged, against 2.4% at 200–300, so the length-band thresholds of Section 5.3 do not equalise the false-alarm rate across lengths. In flight-only mode the human false-alarm rate is unchanged at 4.6%, but macro detection drops to 67.3% and the

adaptive bot is almost invisible (1.9%), because without hold times it cannot be told apart from a human.

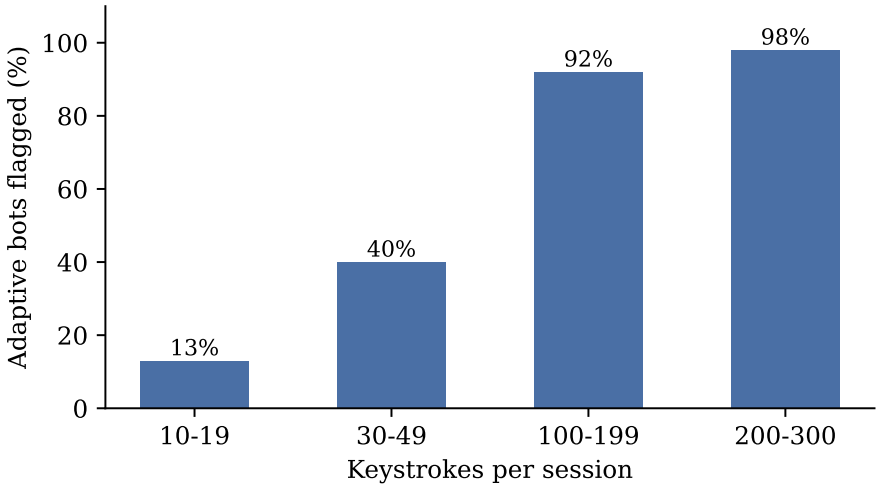


Figure 6.1: Share of adaptive-bot sessions flagged at risk ≥ 50, by session length (full model).

Table 6.3 compares the shipped mixture with the Isolation Forest of the first design, both calibrated to the same false-alarm budget on the same data.

Table 6.3: Detector comparison, full mode, same data and false-alarm budget

Detector	**Adaptive**	**Mimic**	**Macro**	**AUC**
Gaussian mixture (shipped)	54.6%	44.7%	84.9%	0.958
Isolation Forest, PCA-whitened	47.3%	19.4%	82.4%	0.951
Isolation Forest, standardised only	2.4%	4.0%	67.5%	0.897

The comparison shows that preprocessing mattered as much as the choice of detector. The same Isolation Forest that nearly misses the adaptive bot (2.4%) when features are only standardised reaches 47.3% after PCA whitening, close to the mixture. The mixture was kept because it was ahead on every column and because it can be exported as plain parameters.

Limits. When bots are rare a flag is weak evidence: at a 1% share of automated sessions the precision of a risk-50 flag is 15.8%, which is why the output is advisory. A held-out typing style is the larger risk: when one human style was withheld from training and calibration, 51–81% of its sessions were flagged. The model treats the training population as normal, and real deployment would require retraining on real bidders. The scoring latency measured on an AMD64 machine under Windows is a p95 of 0.54 ms in process and 3.4 ms through an in-process ASGI call, within the 15 ms target but excluding network time.

These results support RQ3 for synthetic sessions only. They show that timing separates crude automation from human typing, and that it is much weaker against bots that imitate human timing.

6.4 Ledger Repair (RQ2)

We generated 500 statements of 40 rows for each of three conditions, with integer rupee amounts and a valid recurrence. In the OCR condition one balance digit was replaced by its confusable partner. In the tamper condition one balance was inflated by Rs 5,000–2,00,000. The strict check applies the recurrence with no repair; the lattice check is the engine's row verifier. Table 6.4 shows the number of statements flagged.

Table 6.4: Statements flagged out of 500 per condition

Condition	**Strict recurrence**	**With OCR lattice**
Clean	0	0
Scanning noise (benign)	500	17
Tampered balance	500	500

The lattice cleared 483 of 500 noisy statements (96.6%) and missed none of the 500 tampered ones. Without it, a strict check would send every scanned statement to a human, which is the practical argument for the repair step.

The 17 remaining noise cases were not traced individually; the cause was traced on a separate sample (36 of 1000): the search accepts the first combination that satisfies the recurrence, and sometimes that combination changes the credit or debit and not the corrupted balance. The corrupted balance is then carried forward and the next row fails, so the statement is still flagged, but at the wrong row. This is a weakness in how the repair is attributed. A tampering that happens to equal a confusion swap could be explained away in the same manner, although none of the 500 random inflations was. This result is also easier than it looks: the inflations were drawn at random, so only a small share of them could coincide with a single-digit confusion swap (a 1 changed to a 7 in the thousands place, for example, adds exactly Rs 6,000), and 500 of 500 is close to the expected outcome. A forger who edits with confusable digits was not tested, and that is the case in which the repair step could excuse a real forgery. The simulation is also simple (one error per statement, uniform amounts), so the real rate will differ.

6.5 Cartel Graph (RQ2)

Each of 300 synthetic pools has exactly 12 bidders (66 pairs) with unique directors and assets, an IP chosen from three shared addresses and a bank code from four shared codes, as when firms use a common provider and branches. In 120 pools three random bidders also share one director (a planted ring). A naive method flags every pair sharing any attribute; the engine's `build_cartel_summary` is compared with it. A pool with a planted ring has $66 - 3 = 63$ honest pairs and one without has 66, so the total is $120 \times 63 + 180 \times 66 = 19{,}440$ honest pairs.

The engine never missed a ring, and ring members score about three times higher than other bidders. The reduction in false pairs is modest at the default cap. With 12 bidders over 3

Table 6.5: Cartel experiment at the default cap: 300 pools, 19,440 honest bidder pairs

Measure	Value
Honest pairs flagged, naive method	9,655
Honest pairs flagged, engine	6,771 (29.9% fewer)
Planted rings fully recovered	120 of 120
Mean anomaly score, ring members	0.320
Mean anomaly score, other bidders	0.093

addresses each address is shared by four firms on average, so the cap, which applies only to groups of more than five, affects only the larger random groups and removes only part of the false pairs. The sweep below shows how much of the result depends on that one parameter.

6.5.1 Sensitivity to the public-infrastructure cap

To see how much of the result is a consequence of the cap, we repeated the experiment with the cap set to 2, 3, 4, 5, 6 and 8, using the environment variable `AEGIS_PUBLIC_DEGREE_CAP` and the same pools. The results are in Table 6.6 and Figure 6.2.

Table 6.6: Honest pairs flagged by the engine for different caps (naive method: 9,655)

Cap	Honest pairs flagged	Reduction vs. naive	Rings recovered
2	375	96.1%	120 of 120
3	1,877	80.6%	120 of 120
4	4,199	56.5%	120 of 120
5 (default)	6,771	29.9%	120 of 120
6	8,356	13.5%	120 of 120
8	9,532	1.3%	120 of 120

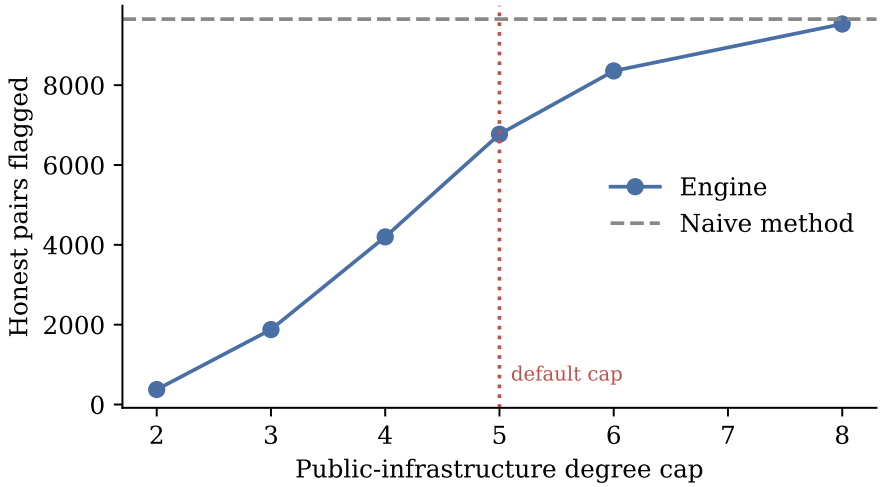


Figure 6.2: False pairs flagged by the engine as the public-infrastructure cap varies.

The 29.9% reduction is therefore a property of the cap, not of the engine. The degree-discounted weight $w(d)$ does not enter the pair flag at all, so it plays no part in this reduction.

A cap of 3 removes about four fifths of the false pairs here, and a cap of 8 removes almost none. Ring recovery stays perfect at every setting, but this must not be over-read. The planted rings are linked by a shared director, and directors are never capped, so lowering the cap cannot hurt recall in this experiment by construction. In real data a ring that shared only an IP address or a bank branch would be hidden by a low cap. The experiment cannot measure that cost, and the right cap has to be chosen on real bidder data, with the trade-off between nuisance flags and missed rings made explicitly. The pair list also still needs the anomaly score to be useful. Even at the default cap the engine flags 6,771 of 19,440 honest pairs (34.8%), and the 29.9% figure is measured against a deliberately naive baseline, so the experiment does not show a useful false-alarm rate on real bidders.

6.6 Trust Optimizer Simulation

The bundled simulation (`run_simulation.py`, seed 42) ranks 20 bidders for one tender. The two colluders have anomaly 0.833 and are both excluded, but not by the collusion penalty: both fail the technical minimum (a technical score of 0.2324 against a threshold of about 0.3), and the penalty μA_i would only have lowered their composite score by about 0.25. In this run the anomaly score therefore did not decide the outcome. Only 7 of the 20 bidders pass the gates, and the winner, B020, has a fused trust of 0.8376 and a price score of 1.17. A bidder with a very low price but the lowest financial score is stopped by the eligibility gate, and a bidder that under-declared litigation receives a legal score of 0.0012. The winner's trust becomes 0.8700 after a successful project and 0.6700 after a failed one. These are constructed cases, not an accuracy measurement. They show the eligibility gate, the legal penalty and the trust update acting on cases built to trigger them. They do not show the collusion penalty μA_i changing an outcome, so that safeguard remains untested in this thesis.

6.7 Tender Pipeline (RQ1)

The project README reports an end-to-end run on a real GeM tender whose extracted EMD, turnover (Rs 27 lakh), experience (three years) and MSE exemption match the document. We did not repeat it for this thesis, since it needs a model key, so it is reported as anecdotal. The unit tests check CDS values, exemption handling and the orchestrator graph. No experiment here measures how often the extractor is wrong or how often the symbolic layer catches the error, so RQ1 is supported by design and tests but not by an error-rate study.

6.8 Discussion and Threats to Validity

The biometrics engine has the most extensive evaluation here (the largest sample, confidence intervals and a baseline comparison), but all of it is synthetic, so none of the engines is validated

on real data. The main threats are:

- **Synthetic data.** Detector and experiment results depend on the generators that the project authors wrote, and the mixture size was chosen on a development set built from the same generator. They show behaviour under stated assumptions, not field accuracy.
- **Own experiments.** We wrote the ledger and cartel experiments to test the safeguards, so they may favour the design. The cap sweep partly guards against this, since it shows a setting where the safeguard looks weak.
- **Untested paths.** The scanned-PDF OCR route and live Gemini extraction were not exercised here, and five tests fail.
- **Single seeds.** The experiments use one seed each. The Wilson intervals describe sampling error within a seed, not variation between generators.

Taken together, the results say something consistent. The safeguards work in the direction intended, although their cost (for example the wrongly cleared forgeries or missed rings they may cause on real data) was not measured, and each one has a parameter or an assumption that decides how much it helps: the cap in the cartel graph, the single-error assumption in the ledger repair, and the session length for the keystroke detector. A reader who wants to rely on any of these engines should treat the parameter as the thing to calibrate first.

Chapter 7

Conclusion and Future Work

7.1 Summary

ProcureGuard-AI is a working platform in which language models read and explain while deterministic code decides. It offers vendors a tender eligibility check with page citations, a late-payment interest calculator, a corrigendum differ and a joint-venture matchmaker. It offers auditors a cartel graph, a bank-statement verifier, a keystroke bot detector and a collusion-resistant trust optimizer. All of these sit behind one role-controlled API, and each engine can be run and tested separately.

The central design decision was where to draw the line between the model and the code. Drawing it at the typed record, so that the model's output is parsed into a schema and nothing after that point is generated, gave a system whose decisions can be replayed and whose figures can be checked against a page. It also meant that a large part of the work was ordinary engineering: statutory formulas, graph weights, accounting identities and calibrated thresholds.

7.2 Answers to the Research Questions

1. **RQ1.** The design makes each decision repeatable and traceable, and unit tests confirm the scoring and exemption logic. The thesis did not measure extractor error rates, so the benefit against hallucination is argued from the design and not shown by an experiment.

2. **RQ2.** In controlled synthetic tests, character-confusion repair cleared 96.6% of noisy statements without hiding any of the 500 tampered ones, and the cartel engine recovered all planted rings. Its false-pair reduction depended heavily on the public-infrastructure cap: 29.9% at the default of 5, 80.6% at 3 and 1.3% at 8. The benefit is therefore real but depends on the data and on a parameter that has to be calibrated. Ring recovery was perfect by construction, since the planted rings share a director and directors are never capped, and the ledger tampering used large inflations that no confusion swap can imitate. The claim that these safeguards do not hide real manipulation is thus only partly tested.

3. **RQ3.** On synthetic sessions the detector flagged 4.6% of humans, caught all regular and burst scripts, but caught only 54.6% of adaptive bots (ROC-AUC 0.958). Timing alone is therefore good against crude automation and weak against bots that imitate human timing. Detection also grows with session length, from 13% at 10–19 keystrokes to 98% at 200–300. The 4.6% false-alarm rate holds only for typing styles seen in training: when one human style was withheld, 51–81% of its sessions were flagged, so the detector must stay advisory.

7.3 Limitations

All evaluation is synthetic. Five backend tests currently fail. Scanned-PDF OCR and live model extraction were not tested here. The fallback parser can return default thresholds when it finds nothing. The payment clock starts at the invoice date. The M2M gateway checks a key but does not verify signed requests. The cartel anomaly score subtracts the partner-sharing term in a way that deserves review, and the trust optimizer's price score is not capped. The corrigendum differ, the compliance score and the payment calculator are checked only by worked examples and unit tests, not by an evaluation. In the differ, a quick check showed that numeric changes can stay above the similarity threshold. In the compliance engine, a medium enterprise would be wrongly exempted, because the exemption rests on a single MSME flag. Each experiment uses a single seed, so variation between generators is not measured.

7.4 Implications

For a public body, the pattern is to keep models on the reading side, make every consequential computation reproducible and cited to a source, and treat behavioural and graph signals as leads for a human reviewer and not as findings. The same pattern would apply outside procurement, in any setting such as benefit eligibility or regulatory filing, where a document has to be read by a model but the decision has legal force. These implications hold only after validation on real data.

7.5 Future Work

1. Repair or update the five failing tests and add a full-journey browser test (login, upload, analyse, display).

2. Measure extractor error rates against a labelled set of real tenders, with and without the symbolic layer, to give RQ1 an empirical answer.

3. Evaluate the cartel and ledger engines on real or realistically scanned data, and choose the public-infrastructure cap on real bidder pools, including rings that share only an IP

address or a bank branch.

4. Retrain the keystroke model on consented real sessions, add sequence-level features against replayed human timing, and compare with published baselines.

5. Start the payment clock from the date of acceptance, keep the bank rate current, and verify signed M2M requests on the server.

6. Replace the fallback defaults with an explicit "not found" result.

7. Repeat the synthetic experiments over several seeds and report the spread.

8. Resolve the sign of the ρ_i term in the cartel anomaly score, cap the trust optimizer's price score at 1, and drop exempted rules from the OCD average.

9. Add an adversarial ledger condition in which the forger edits with confusable digits, a planted ring that shares only an IP address or bank code, and a numeric and date comparison to the corrigendum differ, each with its own experiment.

Appendix A

Engine Constants

Table A.1 collects the constants quoted in Chapters 4 and 5, so that a reader can find them in one place. They are the values in the final repository, not recommendations.

Table A.1: Constants used by the engines

Engine	**Constant**	**Value**
Retrieval	Chunk size / overlap	1500 / 200 characters
Retrieval	Embedding dimension (MiniLM)	384
Retrieval	Chunks per question	4
Extraction	Model temperature	0.1
Compliance	Rule weights (EMD, turnover, experience, certificates)	3, 2, 2, 1
Payment	Bank rate / interest rate	6.5% / 19.5%
Payment	Statutory payment period	45 days
Differ	Similarity threshold for flagging	0.75
Differ	Base scores (restrictive / neutral)	0.3 / 0.1
Differ	Time multiplier τ	3.0 / 1.5 / 1.0
Differ	Verdict thresholds	0.75 / 0.40
Cartel	Public-infrastructure cap	5 bidders
Cartel	Anomaly weights (S_a, S_o, S_c, ρ)	0.45, 0.35, 0.20, -0.30
Ledger	Recurrence tolerance	0.05 rupees
Ledger	Window-dressing Z threshold	3
Ledger	Round-trip window / amount tolerance	7 days / 2%
Ledger	Benford flag threshold	0.25
Keystroke	Clipping of flights / holds	2000 / 1000 ms
Keystroke	Mixture components (full / flight-only)	2 / 4
Keystroke	Risk bands (moderate / high)	50 / 75
Keystroke	Reliability bands (low / medium / high)	< 30 / 30–99 / ≥ 100 keystrokes
Trust	Price blend δ	0.5
Trust	Outlier exclusion for $B_{\min}$	40% from B_{ref}
Trust	Collusion penalty μ	0.3
Trust	Manual-review margin	0.02
Trust	Trust update weights (old / outcome)	0.8 / 0.2
Trust	Beta prior α, β; legal penalty γ	1, 1; 2
Trust	Agent multiplier range η_j	0.85–1.15
Trust	Threshold jitter per tender	± 0.02
Trust	Review trigger (anomaly A)	≥ 0.10
Trust	Score-clustering distance	0.05
Notifier	Scan interval	300 s

Appendix B

API Surface

The gateway registers 14 routers (Section 3.3). Table B.1 groups the gateway areas by the roles that may reach them and is not a one-to-one list of those routers. The exact path prefixes are defined in `backend/src/main.py`, and the access policy is in `backend/src/core/access.py`.

Table B.1: Gateway areas and access

Area	**Who may use it**
Authentication	Anyone (login and registration)
MSME profiles	Read: owning enterprise and reviewers; change: owning enterprise and admin
Tenders, legal diff, orchestrator	Any signed-in user
Matchmaker, notifications, documents	The owning enterprise and reviewers
Vendor bidding	Vendors, scoped to their own enterprise
Cartel graph, solvency	Reviewers only
Behaviour signals and review queue	Set per route; the review queue is for reviewers
Trust optimizer, MCP server (read-only bot-detection tools)	Reviewers only
M2M gateway	Holders of an API key
Background jobs	Any signed-in user

Appendix C

Reproducing the Experiments

The experiment scripts are run from the `ProcureGuard-AI` directory, because they import the engines as packages.

1. **Ledger and cartel experiments.** Run `python ../thesis/experiments/rq2_experiments.py`. The script uses fixed seeds (11 for the ledger, 5 for the cartel pools) and writes `rq2_results.json`.

2. **Cap sweep.** Run `python ../thesis_amrita/experiments/cap_sweep.py`. It sets `AEGIS_PUBLIC_DEGREE_CAP` to each value in turn, re-imports the engine, repeats the cartel experiment on the same pools and writes `cap_sweep.json`. The row for a cap of 5 reproduces Table 6.5.

3. **Figures.** Run `python experiments/make_figs.py` from the `thesis_amrita` directory. Figure 6.2 is drawn from `cap_sweep.json`. Figure 6.1 uses the four length-band values from the biometrics evaluation report.

4. **Keystroke evaluation.** The protocol, seeds and scripts for Section 5.3 are in the biometrics engine under `reports/` and `scripts/`.

References

[1] OECD, "Recommendation of the council on public procurement," tech. rep., OECD Publishing, Paris, 2015.

[2] Ministry of Micro, Small and Medium Enterprises, Government of India, "Public procurement policy for micro and small enterprises (mses) order, 2012," 2012.

[3] Ministry of Finance, Department of Expenditure, *General Financial Rules 2017*. New Delhi: Government of India, 2017.

[4] Government of India, "The micro, small and medium enterprises development act, 2006 (act no. 27 of 2006)," 2006.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[6] K. V. Thai, "Public procurement re-examined," *Journal of Public Procurement*, vol. 1, no. 1, pp. 9–50, 2001.

[7] A. Flynn and P. Davis, "Theory in public procurement research," *Journal of Public Procurement*, vol. 14, no. 2, pp. 139–180, 2014.

[8] N. Dimitri, G. Piga, and G. Spagnolo, eds., *Handbook of Procurement*. Cambridge University Press, 2006.

[9] M. Fazekas and G. Kocsis, "Uncovering high-level corruption: Cross-national objective corruption risk indicators using public procurement data," *British Journal of Political Science*, vol. 50, no. 1, pp. 155–164, 2020.

[10] M. Fazekas, I. J. Tóth, and L. P. King, "An objective corruption risk index using public procurement data," *European Journal on Criminal Policy and Research*, vol. 22, no. 3, pp. 369–397, 2016.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33*, pp. 9459–9474, 2020.

[12] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.

[14] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artificial Intelligence Review*, vol. 56, pp. 12387–12406, 2023.

[15] G. Marcus, "The next decade in AI: Four steps towards robust artificial intelligence." arXiv preprint arXiv:2002.06177, 2020.

[16] J. Wachs and J. Kertész, "A network approach to cartel detection in public auction markets," *Scientific Reports*, vol. 9, p. 10818, 2019.

[17] L. A. Adamic and E. Adar, "Friends and neighbors on the web," *Social Networks*, vol. 25, no. 3, pp. 211–230, 2003.

[18] T. Zhou, J. Ren, M. Medo, and Y.-C. Zhang, "Bipartite network projection and personalized recommendation," *Physical Review E*, vol. 76, no. 4, p. 046115, 2007.

[19] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference*, pp. 11–15, 2008.

[20] F. Benford, "The law of anomalous numbers," *Proceedings of the American Philosophical Society*, vol. 78, no. 4, pp. 551–572, 1938.

[21] M. J. Nigrini, *Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. Hoboken, NJ: John Wiley & Sons, 2012.

[22] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *Journal of the American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.

[23] F. Monrose and A. D. Rubin, "Keystroke dynamics as a biometric for authentication," *Future Generation Computer Systems*, vol. 16, no. 4, pp. 351–359, 2000.

[24] K. S. Killourhy and R. A. Maxion, "Comparing anomaly-detection algorithms for keystroke dynamics," in *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks*, pp. 125–134, 2009.

[25] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[26] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the 8th IEEE International Conference on Data Mining*, pp. 413–422, 2008.

[27] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

[28] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[29] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.

[30] A. Jøsang and R. Ismail, "The beta reputation system," in *Proceedings of the 15th Bled Electronic Commerce Conference*, 2002.

[31] C. W. Cobb and P. H. Douglas, "A theory of production," *American Economic Review*, vol. 18, no. 1, pp. 139–165, 1928.

[32] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[33] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[34] Government of India, "The digital personal data protection act, 2023 (act no. 22 of 2023)," 2023.

[35] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[36] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[37] ProcureGuard-AI project, "Keystroke bot-detection evaluation report." Project repository, `engines/biometrics/reports/evaluation_report.md`, 2026.